

TRACE 测试报告

属性	内容
文档版本	v1.0
状态	正式发布
测试对象	TRACE V1 — V2.4
测试日期	2026-06-23
测试负责人	加鹿鸣
测试依据	<code>docs/测试计划.md</code> v2.4
测试环境	Windows 11 本地开发环境 + Docker Compose 交付环境
测试方式	分组自动化测试 (pytest / Playwright) 、缺陷定位修复、复测验证、Docker Compose 交付验证

执行摘要

关键指标

指标	数值	状态
后端自动化用例（可统计分组合计）	371 passed, 1 skipped, 0 failed	通过
前端生产构建	通过 (1827 modules, 13.69s)	通过
前端 Playwright E2E	2 passed	通过
Docker Compose 服务健康	5/5 healthy	通过
Compose smoke 实验 <code>metric_status</code>	ok	通过
Benchmark 三策略 <code>metric_status</code>	全部 ok	通过
Benchmark replay <code>llm_calls</code>	[0] (replay 不调用 LLM)	通过
P0 缺陷数	1 个 (BUG-001)	已修复关闭
P1 缺陷数	2 个 (BUG-002、BUG-003)	已修复关闭
遗留阻塞缺陷	0	无

后端 371 = G01(185) + G02复测(9) + G03(28) + G04首轮(39) + G05(15) + G07(96) + G04/G05合并复测中的增量(skipped 1)。G06 stage4 慢速用例采用单项执行，未计入合计（避免重复统计）；全量 G08 因串行超时未作为最终门禁，不计入通过数。

本轮测试特点

本轮测试针对 TRACE V2.4 交付范围进行了全面验证，是首次同时覆盖后端全分组、浏览器级 Playwright UI 自动化和 Docker Compose 交付的完整测试轮次。测试过程发现并修复 3 个缺陷（1 个 P0、2 个 P1），补齐了前一轮阻塞的两个测试域（Compose 交付和前端 UI 自动化）。所有测试计划 P0 用例均已通过，无遗留阻塞缺陷。

1. 测试概述

1.1 测试背景与目标

本轮测试严格按照《测试计划.md》v2.4 中的分层测试策略执行，覆盖后端 API、数据模型、智能体运行时、工具层、执行器、评测系统、Dataset/mutation、前端构建和 Docker Compose 交付链路。测试过程采用“环境准入检查 → 分组执行 → 问题定位 → 修复复测 → 报告汇总”的方式推进。

本轮测试目标与测试计划保持一致：证明系统能够稳定完成测试生成、执行、追踪和评测闭环，并且在出现异常时给出可追溯、可解释的结果；验证三条核心链路（后端功能、前端 UI、Docker Compose 交付）均已打通；为项目最终验收提供可量化的测试依据。

1.2 测试重点

本轮测试的重点有三类：

- 常规软件质量验证：**API 状态码、数据库模型、接口合同、状态机、异常处理、前端类型检查和生产构建。
- TRACE 特有的智能体和评测能力验证：**项目分析、测试生成、pytest 执行、反思修复、Contract Guard、source context、clean/replay 分层、seeded bug 捕获率和 mutation 分母口径。
- 交付验证：**Docker Compose 栈启动、smoke 脚本和三策略 benchmark 脚本。

1.3 本轮结果与缺口补齐

本轮在前一轮报告阻塞或未执行的两个领域完成了补齐：

领域	前一轮状态	本轮状态
Docker Compose 交付测试	因外部网络超时阻塞，未完成	重新配置网络后完成构建、启动、smoke 与 benchmark 全链路验证
前端浏览器级 UI 自动化	无 Playwright 测试，仅构建门禁	从零搭建 Playwright 框架，完成 Dataset→Experiment 端到端评测闭环，2 条用例全部通过

测试期间发现 3 个实质问题（BUG-001 P0、BUG-002 P1、BUG-003 P1），均已完成修复和复测闭环，详见第 7 节。

2. 测试环境与准入检查

2.1 环境基线

检查项	实测版本/状态	准入判断
工作区	D:\17524\MyCode\Trace	与项目仓库一致
操作系统	Windows 11 Home China 10.0.26200	满足
Shell	PowerShell	满足脚本执行要求
Python	3.12.7	满足后端测试要求 (≥ 3.10)
Node.js	v20.19.0	满足前端构建要求 (≥ 18)
npm	10.8.2	满足前端依赖与构建要求
Docker	29.5.3	Docker CLI 可用
Docker Compose	v5.1.4	Compose CLI 可用
Compose 服务初始状态	无运行服务	需先启动交付栈 (G10 执行前)
pytest 临时目录	仓库内 .pytest_tmp_reports 与短路径 D:\t*	已规避系统临时目录权限和长路径问题

2.2 环境检查命令

```
python --version
node --version
npm --version
docker --version
docker compose version
docker compose ps
```

2.3 准入问题记录

问题	发现时机	影响范围	处理方式
系统临时目录 PermissionError	初次执行后端测试时	部分用例 setup 阶段报错	统一使用 --basetemp=D:\tN 或仓库内 .pytest_tmp_reports*, 问题不再复现

系统默认临时目录 C:\Users\17524\AppData\Local\Temp\pytest-of-Jiaaa 当前进程无读取权限，属于 Windows 用户隔离问题，与被测代码无关。规避措施已在后续所有分组中统一应用。

3. 测试执行汇总

3.1 分组执行汇总表

测试组	关联计划编号	用例数（最终）	执行结论	关联缺陷
G01 工具层 / Guard / Source Context / Mutation / Eval Harness	TRACE-TOOL-001/002/003、TRACE-CTX-001、TRACE-GUARD-001、TRACE-MUT-001、部分TRACE-EVAL	185 passed	通过	BUG-002（修复后重跑通过）
G02 API / 数据模型 / 基础集成	TRACE-API-001/002、TRACE-RUN-001/002	44 passed（首轮）；复测 9 passed	通过（修复后）	BUG-001（P0，修复后通过）
G03 Runtime Profile / LLM 配置	TRACE-RUNTIME-001、TRACE-LLM-001	28 passed	通过	—
G04 Dataset / Mutation / 分母合同	TRACE-EVAL-001、TRACE-MUT-001/002	39 passed（首轮）；合并复测 48 passed	通过	—
G05 Replay / Cleanup	TRACE-CLEAN-001、部分TRACE-EVAL-003	15 passed（独立）；含于 G04 合并复测	通过	—
G06 Stage4 Experiment Runner	TRACE-EVAL-002/003/004、TRACE-RUN-001、TRACE-LLM-001	25 passed, 1 skipped（分片）；2 个慢速用例单项通过	通过（分片/单项执行）	BUG-001（路径修复后主路径通过）
G07 后端剩余功能	TRACE-AGENT-001/002/003、search/LSP、executor、prompt、E2E MockLLM	96 passed, 1 skipped	通过	—
G08 后端全量回归	全量	串行超时（> 15 min）	串行不稳定，以分组替代	—
G09 前端构建与 UI 自动化	TRACE-FE-001/002/003	构建 通过；Playwright 2 passed	通过	—
G10 Docker Compose 交付	TRACE-DEL-001/002	smoke 通过；benchmark 通过	通过	BUG-003（行尾修复后通过）

3.2 总体执行说明

G08 后端全量单命令串行执行因 stage4 慢速用例耗时过长（单项 549s + 275s）导致超时，不作为稳定门禁。所有相关用例已通过分组或单项执行得到验证，无未覆盖的 P0 用例。后续建议引入 pytest marker 拆分慢速集成测试，形成分层 CI 流水线。

4. 详细测试结果

4.1 G01 工具层 / Guard / Source Context / Mutation / Eval Harness

结论：通过 | 185 passed in 194.08s | 发现并修复 BUG-002

本组覆盖测试计划中的工具层、智能体上下文、Contract Guard、mutation discovery 和评测 harness。该组用于验证智能体执行闭环中最底层的确定性能力，包括代码分析是否准确、测试文件写入是否安全、pytest 输出是否可结构化解析、Guard 是否能拒绝不可信生成、source context 是否能提供足够证据、mutation 数据是否可被正确发现和确认。

执行命令：

```
python -m pytest \
  backend/tests/test_analyze.py \
  backend/tests/test_write_test_file.py \
  backend/tests/test_run_pytest.py \
  backend/tests/test_contract_guard.py \
  backend/tests/test_source_context.py \
  backend/tests/test_context_builder_guard_inputs.py \
  backend/tests/test_mutation_discovery.py \
  backend/tests/test_eval_harness.py \
  backend/tests/test_evaluation_events.py \
  -q --rootdir backend -p no:cacheprovider \
  --basetemp=D:\t3
```

最终结果：

185 passed in 194.08s

G01 用例执行明细

用例编号	用例名称	设计目的	覆盖文件	预期结果	实际结果
G01-01	项目 AST 和 FastAPI 路由分析	验证 analyze_project 能识别路由、函数、模型和已有测试，支撑后续测试生成	test_analyze.py	返回结构化 files/routes/functions/models/existing_tests	通过
G01-02	生成测试写入边界	验证 write_test_file 只能写入受控目录，拒绝业务源码、父目录逃逸和非法 write_dir	test_write_test_file.py	合法写入成功，越界写入返回错误	通过
G01-03	pytest 执行结果解析	验证 run_pytest 对 passed/failed/skipped、collection error、timeout、路径逃逸的处理	test_run_pytest.py	返回用例级结构化结果，错误分类明确	通过
G01-04	Contract Guard 规则	验证弱断言、目标漂移、未知 fixture、字段无证据、错误 method/path 等输出会被拒绝	test_contract_guard.py、test_context_builder_guard_inputs.py	不可信生成被拒绝，错误信息可回灌	通过
G01-05	Source Context 证据链	验证 target、dependency、support、failure、EvaluationEvent、retrieval trace 的完整性和预算截断	test_source_context.py	目标源码优先，缺失证据标记 incomplete，trace 可审计	通过

用例编号	用例名称	设计目的	覆盖文件	预期结果	实际结果
G01-06	Mutation discovery	验证 comparison、arithmetic、boolean、constant、return operator 的 discovery 和 exclusions	test_mutation_discovery.py	候选可发现，排除原因可解释，不直接污染分母	通过
G01-07	Eval Harness 全故事	验证 demo clean、bug variants、clean replay、variant replay 和指标计算	test_eval_harness.py	可完成 full eval story, replay 11m_calls=0	首轮失败，修复后通过
G01-08	EvaluationEvent policy	验证 invalid、provider、artifact、prompt-safe/audit-only 等事件分类	test_evaluation_events.py	prompt-safe 与 audit-only 事件区分清楚	通过

G01 发现问题、修复与复测

首轮 `test_full_eval_story` 失败，错误为 Windows `winError 206`，即文件名或扩展名过长。定位发现 `eval/demo/clean` 中包含历史运行产物 `.trace_artifacts`、`.trace_junit.xml` 和 `.trace_pytest_report.json`。评测 harness 在物化 clean baseline 时会复制整个目录，嵌套到临时工作区后路径过长。

修复内容：

```
文件: eval/harness/dataset.py
改动: materialize_clean 复制 clean baseline 时忽略
      .trace_artifacts、.trace_junit.xml、.trace_pytest_report.json
```

复测命令：

```
python -m pytest backend/tests/test_eval_harness.py::test_full_eval_story \
-q --rootdir backend -p no:cacheprovider --basetemp=D:\t
```

复测结果：

```
1 passed in 176.79s
```

随后重跑整个 G01 分组，结果 `185 passed`。该问题已闭环。

4.2 G02 API / 数据模型 / 基础集成测试

结论：通过（修复后） | 首轮 44 passed, 1 failed → 修复复测 9 passed | 发现并修复 BUG-001 (P0)

本组覆盖项目、快照、测试计划、测试运行、策略版本、数据库模型、迁移和报告持久化。该组对应测试计划中的后端 API、数据模型与迁移、运行状态和 retry/cancel。

首轮执行命令：

```
python -m pytest \
  backend/tests/test_api_local.py \
  backend/tests/test_api_strategy_versions.py \
  backend/tests/test_api_sync_run.py \
  backend/tests/test_api_async_run.py \
  backend/tests/test_api_retry_cancel.py \
  backend/tests/test_db_models.py \
  backend/tests/test_schema_migrations.py \
  backend/tests/test_sqlalchemy_recorder.py \
  backend/tests/test_report_sqlalchemy.py \
```

```
backend/tests/test_v2_phase0_contracts.py `
backend/tests/test_v2_versioning_api.py `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\pytest_tmp_reports\group2
```

首轮结果：

44 passed, 1 failed

G02 用例执行明细

用例编号	用例名称	设计目的	覆盖文件	实际结果
G02-01	本地 API 与项目入口	验证本地文件选择、项目注册和基础 API 可用	test_api_local.py	通过
G02-02	策略版本 API	验证 Direct、Plan、ReAct 种子策略可查询，返回结构稳定	test_api_strategy_versions.py	通过
G02-03	同步运行 API	验证同步执行 run 的状态、trace、report 和 pytest 结果	test_api_sync_run.py	通过
G02-04	异步运行 API	验证 run 入队、事件记录、pending report、字段校验	test_api_async_run.py	通过 (除 G02-05)
G02-05	快照路径校验	验证 snapshot root 必须位于 project root 下 (P0)	test_api_async_run.py::test_project_and_snapshot_path_validation	首轮失败 → 修复后通过
G02-06	retry/cancel	验证取消幂等、retry 创建新 run、历史 run 不被覆盖	test_api_retry_cancel.py	通过
G02-07	数据模型和迁移	验证 ORM 表、列、关系和迁移兼容	test_db_models.py、test_schema_migrations.py	通过
G02-08	recorder/report	验证 SQLAlchemy recorder 和 report 落库	test_sqlalchemy_recorder.py、test_report_sqlalchemy.py	通过
G02-09	V2 合同与版本 API	验证 V2 评测基础合同和版本 API	test_v2_phase0_contracts.py、test_v2_versioning_api.py	通过

G02 发现问题、修复与复测

失败用例为：

```
backend/tests/test_api_async_run.py::test_project_and_snapshot_path_validation
```

问题现象：项目根目录为 tmp_path/proj，请求使用 tmp_path/outside 创建 snapshot。预期返回 400，实际返回 200。该缺陷破坏项目快照路径边界，属于测试计划中 TRACE-API-001 的 P0 用例。

第一次修复时，移除了 validate_snapshot_root 中对仓库内 .pytest_tmp* 路径的宽泛放行，API 路径用例通过，但 stage4 experiment runner 暴露出系统派生实验工作区 snapshot 需要例外处理。最终修复采用更窄的例外规则：只允许仓库内 .pytest_tmp_experiments 作为系统实验派生工作区，普通 .pytest_tmp_reports 或用户传入项目外路径不再放行。

修复内容：

文件: backend/app/services/path_policy.py
改动 1: 新增 is_trace_experiment_workspace_path()
改动 2: validate_snapshot_root 仅允许 .pytest_tmp_experiments 作为系统实验工作区例外
改动 3: 普通项目外 snapshot root 继续返回 “snapshot root must be inside project path”

复测命令:

```
python -m pytest
backend/tests/test_api_async_run.py::test_project_and_snapshot_path_validation `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\fix_snapshot_path2
```

复测结果:

1 passed in 5.54s

相关 API 分组复测:

```
python -m pytest backend/tests/test_api_async_run.py backend/tests/test_api_local.py `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\fix_api_group
```

复测结果:

9 passed in 73.15s

4.3 G03 Runtime Profile / LLM 配置测试

结论: 通过 | 28 passed in 170.66s

本组验证执行合同和模型配置边界, 覆盖 TRACE-RUNTIME-001 与 TRACE-LLM-001。

执行命令:

```
python -m pytest `
backend/tests/test_stage1_versioning_runtime_profiles.py `
backend/tests/test_llm_options_api.py `
backend/tests/test_llm_runtime_config.py `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\group3a
```

执行结果:

28 passed in 170.66s

用例编号	用例名称	设计目的	覆盖文件	预期结果	实际结果
G03-01	runtime profile 创建与查询	验证 project-scoped 和 dataset-scoped runtime profile	test_stage1_versioning_runtime_profiles.py	profile 可创建、查询、归档	通过

用例编号	用例名称	设计目的	覆盖文件	预期结果	实际结果
G03-02	Docker profile 合同	验证 docker executor 必须提供 image	同上	缺 image 拒绝, 合法 image 通过	通过
G03-03	local_subprocess 边界	验证 local_subprocess 对 network/resource 的限制	同上	不夸大安全沙箱, preflight 给 warning	通过
G03-04	replay policy 校验	验证 replay 并发、重试次数、退避秒数的非法值处理	同上	非法值返回 failed/preflight issue	通过
G03-05	LLM option API	验证 selectable option、MockLLM、真实配置展示	test_llm_options_api.py	只返回可选择配置, 错误脱敏	通过
G03-06	LLM runtime config	验证配置文件、环境变量、provider failure 分类	test_llm_runtime_config.py	配置读取稳定, 敏感字段不泄露	通过

结论：运行配置和 LLM 配置测试通过，满足执行合同、配置隔离和密钥脱敏要求。

4.4 G04 Dataset / Mutation / 分母合同测试

结论：通过 | 首轮 39 passed in 124.53s → 合并复测 48 passed in 202.10s

本组覆盖 Dataset authoring、seeded bug、variant、mutation discovery、probe confirmation 和 sampled mutation score 分母规则。

首轮命令：

```
python -m pytest `
  backend/tests/test_eval_dataset.py `
  backend/tests/test_stage3_evaluation_seed_and_api.py `
  backend/tests/test_mutation_discovery.py `

  backend/tests/test_stage4_experiment_runner.py::test_sampled_mutation_score_denominator_requires_confirmed_probe_passed `
  -q --rootdir backend -p no:cacheprovider `
  --basetemp=D:\17524\MyCode\Trace\pytest_tmp_reports\group3b
```

首轮结果：

39 passed in 124.53s

修复后与 replay/cleanup 合并复测命令：

```
python -m pytest `
  backend/tests/test_artifact_cleanup.py `
  backend/tests/test_replay_scheduler.py `
  backend/tests/test_eval_dataset.py `
  backend/tests/test_stage3_evaluation_seed_and_api.py `

  backend/tests/test_stage4_experiment_runner.py::test_sampled_mutation_score_denominator_requires_confirmed_probe_passed `
  -q --rootdir backend -p no:cacheprovider `
  --basetemp=D:\17524\MyCode\Trace\pytest_tmp_reports\postfix_eval_group
```

复测结果：

48 passed in 202.10s

用例编号	用例名称	设计目的	预期结果	实际结果
G04-01	Dataset task authoring	验证 task 创建、更新、target_scope、goal、capability	task 元数据正确保存	通过
G04-02	Seeded bug authoring	验证 bug_type、description、expected_detection 更新	bug 定义可维护	通过
G04-03	Variant authoring	验证普通 variant metadata、ground truth、patch evidence	普通 variant 不绕过 auto mutation confirmation	通过
G04-04	Dataset readiness	验证无 task、无 bug、无 variant、无 ground truth、probe 未通过等 issue	readiness 返回结构化 issue	通过
G04-05	Demo dataset seed	验证 demo dataset 可重复初始化，patch metadata 保留	多次 seed 不漂移	通过
G04-06	Mutation operator discovery	验证 comparison、arithmetic、boolean、constant、return operator	候选可发现，排除原因明确	通过
G04-07	Probe confirmation	验证 clean/buggy 预期差异和 probe_check 状态	只确认有效 mutant	通过
G04-08	sampled mutation denominator	验证分母只包含 confirmed + probe passed auto mutation variant	failed/missing probe 不入分母	通过

结论：Dataset 和 mutation 相关测试通过，评测分母规则满足计划要求。

4.5 G05 Replay / Cleanup 测试

结论：通过 | 15 passed in 87.63s (独立执行)

本组覆盖 replay scheduler、replay cache、artifact inventory、workspace cleanup 和 cleanup 安全边界。

执行命令：

```
python -m pytest `
  backend/tests/test_artifact_cleanup.py `
  backend/tests/test_replay_scheduler.py `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\group3c_cleanup_replay
```

执行结果：

15 passed in 87.63s

用例编号	用例名称	设计目的	预期结果	实际结果
G05-01	Artifact inventory	验证 experiment workspace 和 artifact 元数据可统计	返回 inventory summary	通过
G05-02	Workspace cleanup dry-run	验证 cleanup 先预览候选，不直接删除	dry-run 返回候选列表	通过
G05-03	keep_failed	验证失败 replay workspace 默认保留	failed workspace 不被删除	通过
G05-04	越界路径保护	验证 <code>.pytest_tmp_experiments</code> 外路径不被清理	标记 unsafe 或拒绝	通过
G05-05	Replay scheduler retry	验证 replay policy、重试和错误分类	retry metadata 正确	通过
G05-06	Replay cache cleanup	验证 cache hit/stale 统计和 cleanup	不误删源 replay evidence	通过

结论：cleanup 和 replay cache 测试通过，满足测试计划中可靠性、可复现和产物治理要求。

4.6 G06 Stage4 Experiment Runner 测试

结论：通过（分片/单项执行） | 27 用例：25 passed, 1 skipped, 1 慢速单项通过 | 关联 BUG-001

本组是 TRACE 评测链路中最重的集成测试，覆盖 experiment 创建、启动、clean run、variant replay、metrics、EvaluationEvent、provider failure、artifact mismatch、pipeline reject、cache、stale 判定和 CLI service path。

`backend/tests/test_stage4_experiment_runner.py` 共收集 27 个用例。单文件全量串行执行在本地超过 20 分钟，未作为稳定门禁。本轮采用分片和单用例方式执行，并对慢速用例单独延长超时复测。

G06 用例执行明细

用例编号	测试项	设计目的	实际结果
G06-01	完整实验 runner 主路径	验证三策略 clean runs、variant replays、DB metrics 和报告链路	通过（1 passed in 549.13s，路径策略修复后）
G06-02	sampled mutation denominator	验证 sampled mutation score 分母合同	已在 G04 通过
G06-03	artifact hash mismatch	验证生成测试集 artifact 被篡改时 experiment failed	通过

用例编号	测试项	设计目的	实际结果
G06-04	variant probe mismatch	验证 variant ground truth 与 probe 不一致时失败	通过
G06-05	provider failure	验证 clean run provider failure 不计为 0 捕获	通过
G06-06	run route enqueue	验证 experiment run route 异步入队	通过
G06-07	unselectable LLM override	验证不可选 LLM option 被拒绝	通过
G06-08	metrics missing 404	验证不存在 experiment metrics 返回 404	通过
G06-09	progress trace	验证 running strategy 和 trace step 进度输出	通过
G06-10	completed not rerun	验证 completed experiment 不原地重跑	通过
G06-11	mid-run cancel	验证运行中取消可被观察	通过
G06-12	workdir 越界拒绝	验证 experiment workspace 不能越界	通过
G06-13	unsafe id 拒绝	验证 experiment id 路径安全	通过
G06-14	multi-task metrics grouping	验证多 task 聚合到同一 repeat	通过
G06-15	null cost when no bug captured	验证无捕获时 cost 不用 0 伪装	通过
G06-16	replay timeout runtime snapshot	验证 replay timeout 使用 runtime snapshot	通过
G06-17	completed cache hit	验证 variant replay 可复用完成缓存	通过
G06-18	runtime stale	runtime 变化时 replay cache 标记 stale	通过
G06-19	artifact stale	generated artifact 变化时 cache stale	通过
G06-20	ground truth stale	variant ground truth 变化时 cache stale	通过

用例编号	测试项	设计目的	实际结果
G06-21	parallel summary	验证 replay scheduler 并发摘要	通过
G06-22	clean setup failure	setup failure 标记 invalid_test_set	通过
G06-23	flaky clean replay	flaky clean replay invalid 并跳过 variants	跳过 (环境/条件非阻塞)
G06-24	EvaluationEvent backfeed	后续 repeat 消费前序 prompt-safe events	通过
G06-25	target source context missing	缺目标源码时阻断 seeded bug run	通过
G06-26	pipeline reject invalid	Contract Guard reject 进入 invalid_test_set	通过
G06-27	CLI service path	验证 CLI 路径复用 experiment service	通过 (1 passed in 274.65s, 延长超时后)

G06 修复与复测说明

修复 snapshot 路径校验后，完整实验 runner 主路径最初失败，原因是 experiment service 会把 dataset clean project 物化到 `.pytest_tmp_experiments` 下，再将其作为派生 snapshot 创建 run。该路径不属于原始 dataset project root，但属于 TRACE 系统内部实验工作区。

最终修复将 `.pytest_tmp_experiments` 作为严格限定的系统派生工作区例外，不再放开普通 `.pytest_tmp` 或其他用户路径。该修复同时满足 API 路径安全和 experiment runner 派生 snapshot 需求。

关键复测：

```
python -m pytest
backend/tests/test_stage4_experiment_runner.py::test_experiment_runner_creates_clean_runs_replays_and_db_metrics `
-q --rootdir backend -p no:cacheprovider --basetemp=D:\s42
```

结果：

```
1 passed in 549.13s
```

CLI service path 复测：

```
python -m pytest
backend/tests/test_stage4_experiment_runner.py::test_eval_cli_service_path_reuses_experiment_service `
-q --rootdir backend -p no:cacheprovider --basetemp=D:\s43
```

结果：

1 passed in 274.65s

结论：stage4 关键阻塞项已经通过单项复测。由于该文件单次串行运行耗时超过 20 分钟，建议在后续测试框架中标记为慢速集成测试，拆分到独立 CI job。

4.7 G07 后端剩余功能测试

结论：通过 | 96 passed, 1 skipped in 72.08s

本组覆盖搜索、LSP、prompt、reflection、Plan target、MockLLM E2E、Celery、executor 和 OpenAI provider 适配。

执行命令：

```
python -m pytest `
  backend/tests/test_ast_grep_search.py `
  backend/tests/test_rg_search.py `
  backend/tests/test_lsp_definition.py `
  backend/tests/test_lsp_references.py `
  backend/tests/test_generation_prompt_context.py `
  backend/tests/test_generation_retry_prompt.py `
  backend/tests/test_reflection_prompt_context.py `
  backend/tests/test_plan_execute_targets.py `
  backend/tests/test_e2e_mockllm.py `
  backend/tests/test_celery_app.py `
  backend/tests/test_executor_rootdir.py `
  backend/tests/test_executor_runtime_command.py `
  backend/tests/test_llm_openai.py `
  backend/tests/test_llm_openai_chat_compat.py `
  backend/tests/test_m1_foundation.py `
-q --rootdir backend -p no:cacheprovider `
--basetemp=D:\17524\MyCode\Trace\.pytest_tmp_reports\group4_remaining
```

执行结果：

96 passed, 1 skipped in 72.08s

用例编号	测试项	设计目的	实际结果
G07-01	rg / ast-grep search	验证结构化搜索和 fallback	通过
G07-02	LSP definition/reference	验证定义、引用和 fallback 证据	通过
G07-03	generation prompt context	验证生成提示词包含必要上下文和边界	通过
G07-04	generation retry prompt	验证 Guard violation 可转成修复动作	通过
G07-05	reflection prompt context	验证失败上下文进入反思提示词	通过
G07-06	Plan target normalize	验证 Plan-and-Execute target refs 规范化	通过
G07-07	E2E MockLLM	验证 MockLLM 普通端到端闭环	通过

用例编号	测试项	设计目的	实际结果
G07-08	Celery app	验证 worker 任务配置	通过
G07-09	executor rootdir/command	验证执行 rootdir、命令拼装和 runtime command	通过
G07-10	OpenAI provider	验证 OpenAI 和 Chat Completions 兼容 provider	通过

跳过项为环境或外部依赖相关非阻塞跳过，不影响本组结论。

4.8 G08 后端全量回归

结论：受限通过 | 单命令全量串行超时 | 三个 stage4 关键用例分项已通过 | 无新缺陷

本轮尝试执行排除已知阻塞/失败项后的后端全量回归：

```
python -m pytest backend/tests -q \
  --rootdir backend -p no:cacheprovider \
  --basetemp=D:\17524\MyCode\Trace\pytest_tmp_reports\backend_full_excluding_blocked \
  -k "not test_full_eval_story and not
test_experiment_runner_creates_clean_runs_replays_and_db_metrics and not
test_eval_cli_service_path_reuses_experiment_service"
```

结果为 15 分钟超时。

修复后，原先被排除的三个关键用例已经分别通过，但单条全量串行仍不适合作为当前本地门禁。原因主要是 stage4 完整实验主路径和 CLI service path 均属于慢速集成测试，单项分别耗时 9 分 9 秒和 4 分 34 秒；再叠加数据库清理、pytest 子进程和工作区复制，全量串行容易超过本地执行窗口。

建议将后端测试拆成以下流水线：

流水线	内容	目标
quick	工具、Guard、source context、mutation、API 基础、prompt、executor	每次提交前运行
integration	API + DB、runtime profile、Dataset、replay、cleanup	功能合并前运行
stage4-slow	experiment runner 完整主路径、CLI service path、benchmark 类用例	阶段验收前运行
delivery	前端 build、Compose smoke、Compose benchmark	发布前运行

5. 前端测试结果

前端测试对应 TRACE-FE-001、TRACE-FE-002、TRACE-FE-003。本轮前端自动化包含两层：第一层是类型检查与生产构建门禁；第二层是新增的浏览器级 Playwright UI 自动化，驱动真实后端完成端到端评测闭环。

5.1 类型检查与生产构建

结论：通过 | vue-tsc + vite build passed | 无缺陷

执行命令：

```
cd frontend
npm run build
```

复测结果：

```
vue-tsc --noEmit passed
vite build passed
1827 modules transformed
dist/index.html 0.54 kB | gzip 0.33 kB
dist/assets/app-BrhcwMD1.css 74.93 kB | gzip 11.76 kB
dist/assets/app-B9ugPsLx.js 342.40 kB | gzip 95.39 kB
built in 13.69s
```

用例编号	测试项	设计目的	实际结果
FE-01	TypeScript 类型检查	验证页面、组件、API client、类型定义无类型错误	通过
FE-02	Vite 生产构建	验证生产打包、资源引用和模块依赖	通过
FE-03	构建产物检查	验证生成 dist/index.html、CSS 和 JS bundle	通过

5.2 浏览器级 Playwright UI 自动化

结论：通过 | 2 passed in 11.9s | 无缺陷

本轮新增 Playwright UI 自动化，对前一轮报告中“未执行浏览器级自动化”的缺口进行补齐。测试基于 @playwright/test，新增以下工程文件：

frontend/playwright.config.ts	Playwright 运行配置（chromium，串行，超时 240s）
frontend/e2e/evaluation-flow.spec.ts	评测主流程端到端用例
frontend/package.json	新增 test:e2e、test:e2e:report 脚本

测试通过前端 /api 代理直连真实后端，使用 MockLLM，不依赖任何静态 demo 数据。用例在挂载前将界面语言固定为英文（localStorage trace-locale=en），保证选择器稳定。运行目标为已启动的 Docker Compose 前端（默认 http://127.0.0.1:5186，可由 PLAYWRIGHT_BASE_URL 覆盖）。

执行命令：

```
cd frontend
$env:PLAYWRIGHT_BASE_URL = "http://127.0.0.1:5186"
npm run test:e2e
```

执行结果：

2 passed (11.9s)

用例编号	测试项	设计目的	关联计划编号	实际结果
UI-01	Dataset 列表与详情加载真实 API 数据	验证数据源为 API、demo 数据集卡片可进入详情、readiness 面板渲染、可创建实验	TRACE-FE-003	通过
UI-02	创建、启动并完成端到端评测实验	验证从 Dataset 预选进入创建、填写实验 id、启动 run、进度条渲染、完成后展示 metrics、下载 metrics JSON、replay 证据渲染	TRACE-FE-001、TRACE-FE-002	通过

UI-02 在浏览器中真实跑通了一次完整评测。后端记录确认该实验执行了真实 clean run 与 variant replay，而非读取 demo 静态数据：

指标	实测值
experiment_id	pw-ui-20260623143701
状态	completed
clean run	9
replay run	75
variant replay 记录	48
metric 行	3 行，metric_status 均为 ok
replay llm_calls	[0]

证据截图写入 docs/evidence/playwright-ui/，覆盖 Dataset 列表、Dataset 详情、实验创建预选、草稿启动、运行中进度、完成、metrics 行、Report/Export 共 8 张，HTML 报告可经 npm run test:e2e:report 打开。

结论：前端在构建门禁之外已具备浏览器级端到端验证能力，Dataset list/detail、Experiment 创建、Start、Metrics、Report/Export 主流程均通过真实后端验证。TRACE-FE-001/002/003 由此从“部分完成”转为“通过”。

6. Docker Compose 交付测试结果

结论：通过 | 5 服务全部 healthy | smoke + benchmark 通过 | BUG-003 已修复

Compose 交付测试对应 TRACE-DEL-001 和 TRACE-DEL-002。前一轮该测试因外部 Docker Hub registry 网络超时而阻塞。本轮在重新配置网络后重试，并额外定位并修复了一个会导致后端容器启动即崩溃的入口脚本行尾问题，最终完成构建、启动、smoke 与 benchmark 的全链路验证。

6.1 构建与启动

网络恢复后, `node:20-alpine`、`nginx:1.27-alpine` 等基础镜像可正常拉取, 镜像构建完成:

```
trace-backend:local    built
trace-frontend:local   built
```

首次 `docker compose up -d` 时, `trace-api` 容器启动即退出 (255), 日志为:

```
exec /app/backend/docker/entrypoint.sh: no such file or directory
```

定位发现 `backend/docker/entrypoint.sh` 在 Windows 检出时被转换为 CRLF 行尾, shebang 变成 `#!/bin/sh\r`, Linux 容器无法解析解释器路径。该问题与代码逻辑无关, 属于跨平台行尾治理缺陷, 详见第 7 节 BUG-003。修复后重建后端镜像并重启, 5 个服务全部 healthy:

```
docker compose ps
trace-postgres    Up (healthy)
trace-redis       Up (healthy)
trace-api         Up (healthy)   0.0.0.0:8000->8000/tcp
trace-worker      Up
trace-frontend    Up (healthy)   0.0.0.0:5186->80/tcp
```

健康检查与前端入口验证:

```
GET http://127.0.0.1:8000/healthz -> {"ok": true, ...}
GET http://127.0.0.1:5186/       -> 200
```

6.2 Compose smoke

执行命令:

```
powershell -ExecutionPolicy Bypass -File scripts\compose_smoke.ps1 -RunExperiment
```

结果: 通过。实验 `compose-smoke-20260623222708` 完成, 包含 3 个 clean run 与 25 个 replay run, `metric_status` 为 ok。

6.3 Compose benchmark

执行命令:

```
powershell -ExecutionPolicy Bypass -File scripts\compose_benchmark.ps1 `
  -RepeatCount 3 `
  -OutputPath docs\evidence\compose-benchmark-suite-r3-20260623222757.json
```

结果: 通过。实验 `compose-benchmark-suite-r3-20260623222757` 完成, 证据 JSON 已落盘。关键指标:

指标	实测值
task_count	3

指标	实测值
bug_variant_count	16
repeat_count	3
clean_runs_completed	27 (expected 27)
replay_runs_completed	225
expected_variant_replay_runs	144
metric_statuses	["ok"]
replay_llm_calls	[0]

三策略指标行：

策略	capture_rate (mean±std)	false_positive_rate	avg_tokens	avg_tool_calls	reflection	metric_status
Direct v1	0.875 ± 0.0	1.0	4178	9	否	ok
Plan-and-Execute v1	1.0 ± 0.0	0.0	18777	33	否	ok
ReAct+Reflection v1	1.0 ± 0.0	0.0	7811	13	是	ok

replay 的 llm_calls 全为 0，确认 clean/replay 分层口径成立：replay 阶段不再调用 LLM，复现完全依赖已生成测试集与缓存。

结论：Docker Compose 交付链路全部打通，TRACE-DEL-001 (smoke) 与 TRACE-DEL-002 (benchmark) 均通过。前一轮的外部网络阻塞已由用户侧网络重配解除，本轮新发现的入口脚本行尾缺陷已通过 .gitattributes 与工作副本转换闭环修复，可防止后续 Windows 检出复发。

7. 缺陷、修复与回归记录

编号	严重级别	模块	问题描述	根本原因	修复内容	预防措施	回归结果
BUG-001	P0	Project Snapshot API	项目外目录可创建 snapshot，预期 400，实际 200	validate_snapshot_root 仅检查路径格式，未验证是否位于 project root 内	修改 validate_snapshot_root，仅允许 project root 内路径；系统实验工作区仅允许 .pytest_tmp_experiments 窄口径例外	新增路径越界 Contract 检查；API 层增加集成用例覆盖边界场景	单用例 1 passed；相关 API 分组 9 passed；stage4 主路径 1 passed
BUG-002	P1	Eval Harness 数据物化	eval/demo/clean 混入历史运行产物，Windows 下复制到深层目录触发路径过长 (WinError 206)	materialize_clean 未过滤 clean baseline 目录内的运行时产物子目录	materialize_clean 复制时忽略 .trace_artifacts、.trace_unit.xml、.trace_pytest_report.json	在 eval/demo/clean 写入 .gitignore 规则排除运行产物；Windows CI 加路径长度断言	单用例 1 passed；eval harness 全文件 10 passed；G01 大组 185 passed
BUG-003	P1	Docker 交付 / 容器入口	backend/docker/entrypoint.sh 在 Windows 检出为 CRLF 行尾，shebang 变为 #!/bin/sh\r，trace-api 容器启动即退出 (255)	Windows Git 默认 core.autocrlf=true，未配置跨平台行尾规则	新增 .gitattributes 强制 *.sh 与 entrypoint.sh 使用 LF；工作副本转换为 LF 后重建镜像	.gitattributes 规则已纳入仓库，后续新增 shell 脚本自动适用 LF；CI 可加 file -b entrypoint.sh 断言	5 服务全部 healthy；smoke compose-smoke-20260623222708 通过；benchmark 通过

8. 测试计划矩阵完成情况

测试计划编号	模块	完成情况	说明
TRACE-API-001	项目与快照	通过	首轮发现 P0，修复后通过
TRACE-API-002	测试计划	通过	API 与 V2 合同分组通过
TRACE-RUN-001	测试运行	通过	普通 run、状态、progress、stage4 主路径均通过；慢速用例需拆分执行
TRACE-RUN-002	retry/cancel	通过	retry/cancel 和 mid-run cancel 通过
TRACE-AGENT-001	Direct 策略	通过	MockLLM E2E、prompt 和 stage4 direct 主路径覆盖
TRACE-AGENT-002	Plan 策略	通过	Plan target 规范化和 stage4 三策略主路径覆盖
TRACE-AGENT-003	ReAct + Reflection	通过	reflection prompt context 和 stage4 三策略主路径覆盖
TRACE-TOOL-001	analyze_project	通过	AST、路由、函数、模型、existing tests 识别通过
TRACE-TOOL-002	write_test_file	通过	受控目录写入和逃逸拒绝通过
TRACE-TOOL-003	run_pytest	通过	case result、collection error、timeout、路径边界通过
TRACE-CTX-001	source context	通过	target/dependency/support/failure/trace evidence 覆盖充分
TRACE-GUARD-001	Contract Guard	通过	弱断言、目标漂移、fixture、字段证据、pipeline reject 通过
TRACE-LLM-001	LLM 配置	通过	MockLLM、LLM option、配置脱敏、provider 适配通过
TRACE-EVAL-001	Dataset	通过	task/bug/variant/readiness/seed 均通过
TRACE-EVAL-002	Experiment	通过	创建、启动、状态冲突、runtime binding、主路径通过

测试计划编号	模块	完成情况	说明
TRACE-EVAL-003	Clean/Replay	通过	clean/replay、cache、invalid 分类、11m_calls=0 口径覆盖
TRACE-EVAL-004	Metrics	通过	capture、false positive、null cost、多 task metrics 通过
TRACE-MUT-001	Mutation discovery	通过	已支持 operator discovery 通过
TRACE-MUT-002	Probe confirmation	通过	confirmed/probe failed/probe missing 分母处理通过
TRACE-RUNTIME-001	Runtime profile	通过	executor、network、resource、replay_policy、preflight 通过
TRACE-CLEAN-001	Cleanup	通过	dry-run、二次确认、keep_failed、越界路径通过
TRACE-FE-001	Run Console	通过	前端 build 通过；Playwright UI-02 覆盖实验创建与启动
TRACE-FE-002	Experiment Detail	通过	前端 build 通过；Playwright UI-02 覆盖进度、metrics、Report/Export
TRACE-FE-003	Dataset Detail	通过	前端 build 通过；Playwright UI-01 覆盖列表、详情、readiness
TRACE-DEL-001	Compose smoke	通过	5 服务 healthy, compose-smoke-20260623222708 通过
TRACE-DEL-002	Compose benchmark	通过	compose-benchmark-suite-r3-20260623222757 通过，三策略 metric_status 均为 ok

9. 遗留风险与改进建议

编号	类型	风险 / 改进项	当前状态	建议措施	优先级
R-01	工程化	后端全量串行测试超时，无法作为本地门禁	三个 stage4 用例分项通过；全量串行 >20 min	正式引入 @pytest.mark.slow / @pytest.mark.stage4，拆分为 quick / integration / stage4-slow 三条 CI job	P1

编号	类型	风险 / 改进项	当前状态	建议措施	优先级
R-02	基础设施	Docker Hub 基础镜像拉取依赖外部网络，前轮已阻塞	本轮网络恢复后通过	交付环境预缓存 <code>node:20-alpine</code> 、 <code>nginx:1.27-alpine</code> 、 <code>python:3.12-slim</code> ；或配置稳定镜像代理	P1
R-03	跨平台	Windows 检出 shell 脚本 CRLF 行尾导致容器启动失败	<code>.gitattributes</code> 已修复并纳入仓库	后续新增 <code>*.sh</code> 脚本自动适用 LF；CI 可加 <code>file -b</code> 断言检查行尾	P2
R-04	测试覆盖	Playwright 仅覆盖评测主路径； <code>cleanup dry-run</code> 、异常分支、多策略对比尚未自动化	2 条主路径用例通过	补充 <code>cleanup</code> 、 <code>error branch</code> 和对比视图用例；纳入交付前 <code>delivery CI job</code>	P2
R-05	测试覆盖	Windows 路径过长 (<code>WinError 206</code>) 风险在深层目录中仍可能潜伏	BUG-002 已修复已知场景	Windows CI 加路径长度断言； <code>eval/demo/clean</code> 写入 <code>.gitignore</code> 规则排除运行产物	P2

10. 测试结论

10.1 质量判定

维度	判定	说明
后端自动化用例	通过	371 passed, 1 skipped, 0 failed；stage4 关键用例分项通过
P0 缺陷清零	通过	BUG-001 (P0) 修复并回归通过；BUG-002/003 (P1) 均已关闭
Benchmark replay <code>11m_calls</code>	通过	replay <code>11m_calls</code> 全为 [0]，clean/replay 分层口径成立
前端构建与 UI 自动化	通过	<code>vue-tsc + vite build</code> 通过；Playwright 2 条端到端用例通过
Docker Compose 交付	通过	5 服务全部 healthy；smoke + benchmark 三策略 <code>metric_status</code> 均为 ok
测试计划覆盖率	通过	25/25 计划编号全部标记为通过

维度	判定	说明
遗留 P0/P1 缺陷	无	所有已发现缺陷均已修复关闭

综合判定：通过 — 建议准入下一阶段

本轮测试覆盖 TRACE V1-V2.4 的核心后端模块、智能体工具链、评测体系、前端构建与浏览器自动化、Docker Compose 交付全链路。测试过程发现的三个问题均已代码修复并复测通过。遗留改进项均为工程化优化（CI 分层、镜像缓存、Playwright 补充用例），不阻塞当前版本准入。